CM-EDU-003 / Student reference

CogniCar API

Read vehicle state and sensors, send commands, and record your experiment. C++17 and Python 3 expose the same `Cognicar` object and method names. ROS 2 topics are listed in the final two sections.

1 Connect and read

Source `env.sh` and start the simulator or car bridge as described in *Getting Started*. These examples read one fresh model sample without driving. Both wait up to five seconds, then shut down. These settings use ROS simulation; see page 3 for direct UDP and real-car settings.

Python 3

```python
import time
import cognicar_interface as ci
cfg = ci.Config()
cfg.backend = ci.Backend.Ros
cfg.simulation = True
car = ci.Cognicar()
if not car.Initialize(cfg): raise SystemExit("Initialization failed")
try:
    deadline = time.monotonic() + 5.0
    while time.monotonic() < deadline:
        model = car.LatestModel()
        if model is not None:
            print(model.pose.position.x, model.twist.linear.x)
            break
        car.WaitForUpdate(50)  # milliseconds
    else: raise SystemExit("No fresh model feedback")
finally:
    car.Shutdown()
```

C++17

```cpp
#include <CognicarInterface/CognicarInterface.hpp>
#include <chrono>
#include <iostream>
int main() {
    namespace ci = cognicar_interface;
    using namespace std::chrono;
    ci::Config cfg;
    cfg.backend = ci::Backend::Ros;
    cfg.simulation = true;
    ci::Cognicar car;
    if (!car.Initialize(cfg)) return 1;
    ci::ModelFeedback model;
    bool ready = false;
    const auto deadline = steady_clock::now() + seconds(5);
    while (steady_clock::now() < deadline) {
        if (car.LatestModel(model)) { ready = true; break; }
        car.WaitForUpdate(milliseconds(50));
    }
    if (ready) std::cout << model.pose.position.x << '\n';
    car.Shutdown();
    return ready ? 0 : 2;
}
```

2 Read telemetry correctly

Python getters return a value or `None`. C++ getters return `bool` and fill the supplied output argument only on success. Missing or expired feedback, or a stopped backend, makes the getter fail. The default maximum age is **250 ms**.

Getter / returned type	Useful fields and units
`LatestModel` / `ModelFeedback`	`pose`: position (m), quaternion in map. `twist`: linear (m/s), angular (rad/s) in body/`base_link`. `beta`: sideslip (rad).
`LatestWheelEncoder` / `WheelEncoderFeedback`	Pass `Motor.FrontLeft` or `FrontRight` in Python; `Motor::FrontLeft` or `FrontRight` in C++. `speed_rpm`; direct feedback also carries `position_rad` and fault flags.
`LatestMotorFeedback` / `MotorFeedback`	Rear-left or rear-right motor: `speed_rpm`, `current_A`, `temperature_C`, `fault_flags`. Direct feedback also carries `torque_mNm`, `voltage_mV`.
`LatestServoFeedback` / `ServoFeedback`	`angle_rad`; direct feedback also carries `rate_rads`, `temperature_C`, `fault_flags`.
`LatestImu` / `ImuData`	`linear_acceleration_ms2`, `angular_velocity_rads`: three-axis vectors. Direct feedback also carries magnetic field (mgauss), temperature and faults.
`LatestLocalization` / `LocalizationFeedback`	Accepted LiDAR map pose, 6x6 `covariance` and freshness `sample`; both backends.
`LatestLidarScan` / `LidarScan`	`ranges`: binned distances (m). `points`: polar returns with `angle_rad`, `distance_mm`. Angular metadata in rad; scan time in s.

Freshness is per sample

Python 3: a stricter age limit

```
model = car.LatestModel(max_age_ms=100)
if model is not None:
    print(model.sample.sequence, model.sample.age_s)
```

For C++, pass `std::chrono::milliseconds(100)` after the output argument.

Use `sample.AgeSeconds()` and `sample.IsFresh()` in C++; Python exposes `sample.age_s` and `sample.IsFresh()`. Age measures local receipt time on a steady clock, not sensor acquisition time.

`IsRunning()` only describes the local backend. `IsConnected()` means recent feedback has arrived; it does not prove every sensor is fresh. `WaitForUpdate` waits for an update, not a complete synchronized sensor set. Check every input used by your controller on every cycle and stop issuing motion when a required input expires.

> **Availability** The simulator provides model, wheel, rear-motor and IMU feedback, but no servo feedback or LiDAR. ROS carries only the fields represented by its topics; other struct fields remain defaults and must not be treated as measurements. With real-car LiDAR enabled, `LatestModel()` requires accepted SLAM within 500 ms; tracking uses the fused model pose.

3 Send commands and manage a session

Method (same spelling in both languages)	Meaning
`SetRearSpeedRpm(left, right)`	Rear wheel speed requests (RPM); positive values request forward wheel rotation.
`SetRearCurrentA(left, right)`	Signed motor currents (A). Forward drive uses left positive, right negative.
`SetSteeringAngleRad(angle)`	Front steering request in rad; positive turns left.
`SetLidarEnabled(enabled)`	Start or stop LiDAR and built-in SLAM.
`SetLeds(leds)`	Indexed RGB values, up to 100 LEDs. Python: list of `ci.Rgb(r,g,b)`; C++: `std::vector<ci::Rgb>`. Channels: 0–255.
`SetLedMode(mode, mask, value)`	Select a `LedMode`; optional control bits for daylights, tail, left, right and brake.

Commands return `bool`. Check it: success means the interface accepted/sent the request, not that the vehicle reached the target. Rear-left and rear-right commands are separate sends; a failed pair is not an atomic rollback.

Choose either RPM or current control for a run. Do not let multiple programs publish competing commands. The supplied `drive` example uses RPM; `autopilot` uses the low-level controller's signed current outputs.

Backend and lifecycle

Set `cfg.backend` explicitly: `Backend.Ros` in Python or `Backend::Ros` in C++. The API's default is `DirectUdp`. For direct UDP, also set `udp_ip` and `udp_interface` to your target; `simulation = true` selects the loopback simulator instead. For the page 1 examples, change `Ros` to `DirectUdp` for a direct SDK. Set `cfg.simulation` to `False` in Python or `false` in C++ for the real car. A ROS backend requires a ROS-enabled SDK.

ROS spins in the background by default. If you set `spin_ros_background = false`, call `SpinOnce()` regularly. `SetCyclicMode(true)` resends remembered commands; it is off by default. It does not replace your controller's input-freshness checks. `Shutdown()` stops cyclic mode, requests zero rear RPM and centred steering, and closes the backend. A stop request still depends on a working connection.

Command readback and CSV logging

`LatestMotorSpeedRequestRpm` reads a cached RPM request, not measured speed. Unlike telemetry getters it has no age argument. In ROS it can observe requests published by other nodes; use wheel or motor feedback to measure motion.

`EnableLogging(mask, path, rate_hz)` starts periodic CSV logging; `DisableLogging()` stops it. Defaults log pose and speed at 100 Hz; the maximum logging rate is 500 Hz. `AddLogVar(name, mask)` registers a custom column and `AddLogValue(mask, value)` updates it. Use a distinct user bit starting at `kLogFirstUserMask` for each variable. Sampling the CSV faster does not increase sensor update rates.

To build the page 1 C++ example, save it as `exercises/read_model.cpp`; run `cmake -S . -B build`, then `cmake -build build -target read_model`. Run `./build/read_model`. Save the Python example as `read_model.py` and run it with `python3 read_model.py`.

4 ROS 2: command topics

Your node publishes these topics; the car's `ros_concentrator` bridge subscribes. Use the same ROS domain and compatible DDS networking as the bridge. The simulator supports the motion loop, with the sensor limitations listed on page 2.

Topic	Message type / payload
`/powertrain/rl/rpm_req` `/powertrain/rr/rpm_req`	`std_msgs/Float64`: rear wheel speed (RPM).
`/powertrain/rl/current_req` `/powertrain/rr/current_req`	`std_msgs/Float64`: signed motor current (A).
`/servo/front/angle_req`	`std_msgs/Float64`: steering (rad).
`/gps/pose`	`geometry_msgs/PoseStamped`: externally supplied body pose, e.g. from localization. Use the map frame.
`/led_cmd`	`std_msgs/UInt8MultiArray`: `[count, r,g,b, ...]`.
`/led_mode_cmd`	`std_msgs/UInt8MultiArray`: `[mode, mask, value]`.
`/lidar/command`	`std_msgs/UInt8`: 0 disables, 1 enables. Reliable, transient-local QoS.

`rl`/`rr` mean rear-left/rear-right; `fl`/`fr` mean front-left/front-right. Message types are abbreviated as `package/Type` in the tables; ROS CLI and code use `package/msg/Type`.

LED modes

Value	Mode
0, 1, 2, 3	Off, Car, Rainbow, Police
4, 5, 6	WelcomeHomeStart, WelcomeHomeStop, Manual

The control mask selects which lights to change; the value supplies their on/off bits. Bits 0–4 are daylights, tail, left indicator, right indicator and brake.

Inspect before connecting your controller

ROS terminal

```
ros2 topic list
ros2 topic info /state/odometry --verbose
ros2 topic echo /state/odometry --once
```

Use one owner for motion commands and publish at your control-loop rate. **Steering is in radians**, including requests and feedback. Current signs differ from wheel-RPM signs because the right motor is mirrored. There is no public C++/Python pose-injection method; use `/gps/pose` if your ROS localization node supplies an external pose.

5 ROS 2: feedback topics

Your node subscribes; the car bridge publishes when matching UDP feedback arrives. Most bridge feedback uses reliable, volatile QoS with depth 1. LiDAR uses **SensorData QoS** (best effort); choose a compatible subscriber profile.

Topic	Message type / meaning
`/state/odometry`	`nav_msgs/Odometry`: coherent model pose and twist. Preferred state topic.
`/map/pose`	`geometry_msgs/Pose`: legacy, unheadered map pose.
`/map/twist`	`geometry_msgs/Twist`: legacy, unheadered body-frame twist.
`/map/side_slip_angle`	`std_msgs/Float64`: body sideslip (rad).
`/powertrain/rl/rpm_fb` `/powertrain/rr/rpm_fb`	`std_msgs/Float64`: rear wheel speed (RPM).
`/powertrain/rl/current_fb` `/powertrain/rr/current_fb`	`std_msgs/Float64`: rear motor current (A).
`/powertrain/rl/status` `/powertrain/rr/status`	`std_msgs/Int16`: rear motor fault flags.
`/powertrain/rl/temp` `/powertrain/rr/temp`	`std_msgs/Float64`: rear motor temperature (°C).
`/powertrain/fl/wheel_enc` `/powertrain/fr/wheel_enc`	`std_msgs/Float64`: front wheel encoder speed (RPM).
`/servo/front/angle_fb`	`std_msgs/Float64`: front steering (rad).
`/imu`	`sensor_msgs/Imu`: acceleration (m/s^2), angular velocity (rad/s).
`/scan`	`sensor_msgs/LaserScan`: 720 angular bins; ranges in m, angles in rad.
`/lidar/points`	`sensor_msgs/PointCloud2`: full returns, up to 1440 points; XYZ in m in the LiDAR frame, $z = 0$.

`SetLidarEnabled(true)` starts embedded SLAM; accepted corrections feed `/gps/pose` and `LatestLocalization()`. Calibrated defaults are compiled in; no YAML is needed. Set `Config.simulation=true` for simulator GPS, or `localization_map` for a saved map. `Shutdown()` stops SLAM and the sensor.

Frames, timestamps and missing data

Odometry pose uses `map`; twist uses `base_link` at the vehicle centre of gravity. Bridge frame names are configurable. Its timestamp is bridge publication time. Covariance arrays are zero-filled because the UDP model feedback has no covariance; zeros are not a claim of perfect certainty.

The scalar feedback and legacy pose/twist messages have no headers. Track receipt age in your own ROS node and handle loss of required inputs. Raw ROS subscriptions do not automatically apply the SDK's 250 ms expiry checks.

> **Exact interface** See the bundled `TOPICS.md` and `include/CognicarInterface/CognicarInterface.hpp` for the complete contract. This guide reflects the source interfaces reviewed on 28 September 2026.